Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Carrera de Ciencias y Sistemas

Segundo Semestre 2026

PROYECTO FINAL

SISTEMAS OPERATIVOS 2

REDXela · Centro de Distribución

MANUAL TÉCNICO

Catedrático: Ing. Otto Soto

Estudiante: Miguel Angel Quemé Gómez

Carné: 202031398

Quetzaltenango, 27 de octubre de 2026

Contenido

1. Infraestructura **3**
1.1 Estructura del código 3
1.2 Arquitectura 3
1.3 Hilos del sistema 4
2. Mapa de conceptos del curso **5**
3. Recursos y exclusión mutua **5**
4. Tipos de mercancía **5**
5. Planificación **6**
6. Comunicación Recepción ↔ Inventario **7**
7. Almacenamiento **8**
8. Deadlock **9**
8.1 Condiciones de Coffman 9
8.2 Prevención (modo normal) 9
8.3 Provocar el deadlock 9
8.4 Detección 10
8.5 Recuperación 11
9. Manejo de excepciones **12**
10. Generador de pedidos **12**
11. Pruebas **12**

1. Infraestructura

Elemento	Tecnología
Lenguaje y plataforma	Java 17
Marco de trabajo	Spring Boot 3.4 (solo para servir la API REST y los archivos estáticos)
Construcción	Maven
Interfaz	HTML + CSS + JavaScript sin librerías, consulta `/api/state` cada 500 ms
Almacenamiento	Solo memoria (sin base de datos)
Pruebas	JUnit 5

Decisión: sin base de datos. El objetivo del curso es la concurrencia y la sincronización. Una base de datos añadiría una segunda capa de control de concurrencia (transacciones) que ocultaría los mecanismos que sí se evalúan. Como contrapartida, el estado se pierde al reiniciar.

Decisión: sondeo (polling) en lugar de WebSocket. Para una interfaz de observación, una foto del estado cada 500 ms es suficiente, más simple y más robusta. La lógica del simulador corre en sus propios hilos y no depende de que haya un navegador conectado.

1.1 Estructura del código

```
com.redxela
├── model        Order, OrderItem, Product, Customer, ServiceLevel, ResourceType,
│                MerchandiseType, OrderStatus
├── core         ResourceManager, DistributionCenter, SchedulingPolicy, Warehouse,
│                InventoryControl, ReceptionArea, OrderGenerator, StatsService,
│                Catalog, CustomerRegistry, EventLog, ArrivalMessage
├── exception    RedxelaException, WarehouseFullException
└── web          ApiController (REST y manejo de excepciones)
```

1.2 Arquitectura

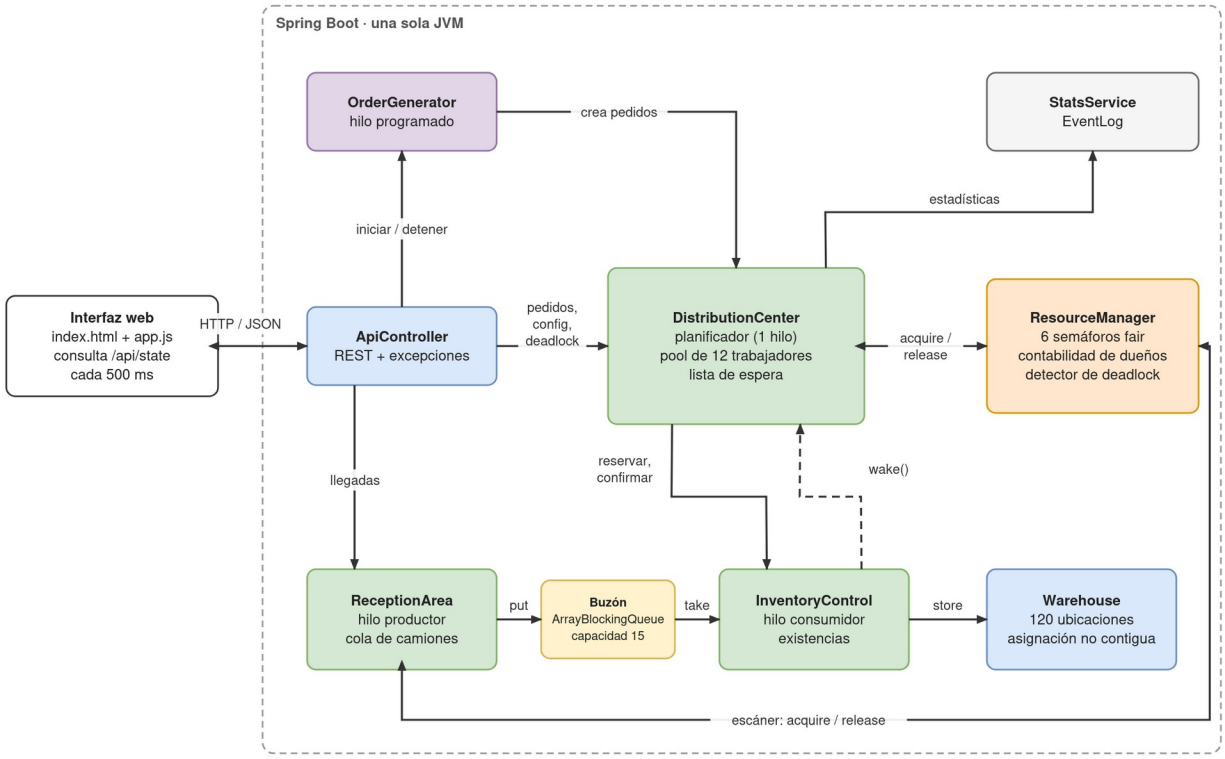


Figura 1. *Arquitectura general del sistema*

Los diagramas de este manual están disponibles como archivos `.drawio` editables en la carpeta `docs/diagramas`.

1.3 Hilos del sistema

Hilo	Cantidad	Función
`planificador`	1	Elige el siguiente pedido de la lista de espera según la política
`trabajador`	12 (pool fijo)	Cada uno procesa un pedido completo
`recepcion-productor`	1	Toma camiones, usa el escáner y envía mensajes a Inventario
`inventario-consumidor`	1	Procesa los mensajes del buzón
`generador`	0 o 1	Genera pedidos y camiones automáticamente
Hilos HTTP de Tomcat	varios	Atienden la interfaz

Los hilos son daemon y se interrumpen en `@PreDestroy`, por lo que la aplicación termina limpiamente.

2. Mapa de conceptos del curso

Requisito del enunciado	Concepto	Implementación
Recursos limitados (6, 2, 10, 3, 4, 1)	Semáforos contadores	`Semaphore(n, true)` por tipo de recurso
Escáner compartido Recepción/Despacho	Exclusión mutua	`Semaphore(1)`, pedido por ambos
Qué pedido se atiende primero	Planificación	`SchedulingPolicy` (3 `Comparator`)
Muchos pedidos a la vez	Hilos y paralelismo	`ExecutorService` de tamaño fijo
Recepción ↔ Inventario	Comunicación entre procesos (productor-consumidor)	`ArrayBlockingQueue` acotada
Almacén con ubicaciones no contiguas	Administración de almacenamiento	`Warehouse` (asignación no contigua)
Bloqueo mutuo	Deadlock: prevención, detección y recuperación	`ResourceManager` y `DistributionCenter`

3. Recursos y exclusión mutua

Cada `ResourceType` es un `java.util.concurrent.Semaphore` cuyo valor inicial es su capacidad.

Por qué semáforo contador: un recurso con *n* unidades idénticas se modela naturalmente con un semáforo en *n*; `acquire()` decrementa y bloquea en cero, `release()` incrementa y despierta a un hilo en espera. Con capacidad 1 (escáner) se comporta como un mutex.

Por qué `fair = true`: los hilos en espera se atienden en orden FIFO, lo que evita que un hilo quede esperando indefinidamente (inanición) cuando otros siguen llegando.

Contabilidad propia. Un `Semaphore` solo cuenta permisos; no sabe quién los tiene. Para mostrar "en uso por quién" y poder detectar deadlock, `ResourceManager` mantiene, bajo un monitor (`synchronized (lock)`):

- `holders`: recurso → lista de dueños (`PED-12`, `RECEPCION`).
- `waiting`: dueño → recurso que espera.
- Integral de ocupación por recurso, para la ocupación media.

El orden de operaciones es deliberado: al **liberar**, primero se actualiza la contabilidad y después se libera el permiso; al **adquirir**, se registra la espera antes de bloquearse y el dueño después de obtener el permiso. Así, las ventanas de inconsistencia solo hacen que el detector sea conservador (puede tardar un ciclo en ver un deadlock, pero nunca reporta uno falso).

4. Tipos de mercancía

Cada tipo requiere una **combinación distinta** de recursos, pedidos en el orden natural indicado:

Tipo	Recursos (orden natural)
Frágil	Encargados → Empaque → Calidad
Pesado	Montacargas → Encargados → Carga
Voluminoso	Montacargas → Empaque → Carga
Electrónico	Escáner → Carga → Empaque
Perecedero	Carga → Escáner → Calidad

Escenario de contención obligatorio: *Electrónico* y *Perecedero* comparten el **escáner** y las **áreas de carga**, pero los piden en **orden inverso**. Dos pedidos de estos tipos en momentos cercanos disputan los mismos dos recursos.

Un pedido puede contener productos de varios tipos; sus recursos son la unión de los de cada tipo (sin repetir).

5. Planificación

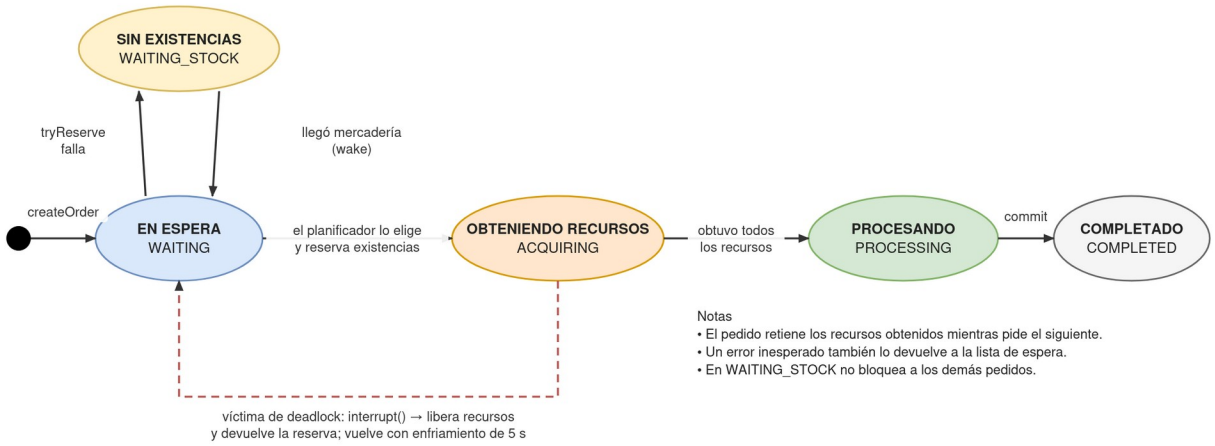


Figura 2. *Ciclo de vida de un pedido*

La lista de espera contiene los pedidos que aún no tienen hilo. El hilo **planificador** espera un trabajador libre (`workerSlots`, un semáforo con el tamaño del pool) y entonces elige el pedido mínimo según la política activa. La política es un `Comparator`, por lo que **se cambia en caliente**.

La cantidad de unidades del pedido es la medida del trabajo: el tiempo de procesamiento es `unidades × 40 ms`, acotado entre 1.5 s y 8 s.

Política	Criterio	Ventaja	Riesgo
Prioridad estricta	Nivel, luego llegada (FCFS)	Cumple el nivel contratado	Inanición de niveles bajos

Política	Criterio	Ventaja	Riesgo
Con envejecimiento	`nivel − minutos_esperados / 15`	Un pedido viejo termina pasando a los nuevos; sin inanición	Mezcla prioridades
Menor cantidad primero (SJF)	Unidades, luego nivel	Reduce la espera media (SJF es óptimo en tiempo medio de espera)	Pedidos grandes pueden esperar mucho

Estadísticas por política. Cada pedido completado se acumula en la política que estaba activa cuando fue despachado, para poder compararlas. *Espera* = desde la creación hasta que obtiene todos sus recursos. Un pedido está *fuera de objetivo* si esa espera supera el máximo de su nivel (Exprés se toma como 3 s).

Pedidos sin existencias. Antes de despachar, el planificador intenta reservar atómicamente todas las unidades. Si no hay, el pedido se marca `WAITING_STOCK` y se salta, de modo que no bloquea a los demás (se evita el bloqueo de cabeza de línea). Cuando llega mercadería, Inventario avisa al planificador (`stockListener`) y se reevalúan.

Limitación conocida. La política decide qué pedido recibe un hilo. Una vez que el pedido espera un recurso concreto, los semáforos lo atienden en orden FIFO, no por prioridad.

6. Comunicación Recepción ↔ Inventario

Las dos áreas **no comparten estructuras**; solo se comunican por mensajes (`ArrivalMessage`: producto, cantidad, instante) a través de un buzón `ArrayBlockingQueue` de capacidad 15.

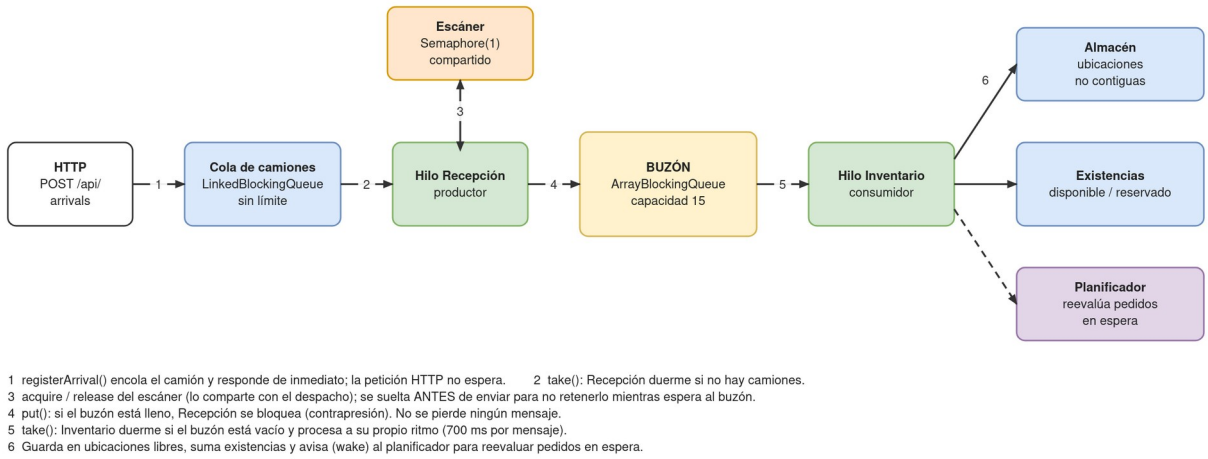


Figura 3. *Comunicación Recepción → Inventario (productor-consumidor)*

- **Sin pérdida de datos:** `put()` bloquea al productor cuando el buzón está lleno (**contrapresión**) en lugar de descartar mensajes.
- **Sin bloqueo indefinido:** el consumidor siempre avanza (su retraso es finito: 700 ms por mensaje), así que el productor se desbloquea en cuanto se libera un espacio.
- **Sin espera activa:** `take()` suspende al consumidor cuando no hay mensajes.

- **La llamada HTTP no espera:** los camiones se encolan en una cola propia no acotada y la petición regresa de inmediato.
- Recepción **suelta el escáner antes** de hacer `put()`, para no retenerlo mientras espera por el buzón.

El retraso de 700 ms en Inventario es intencional: permite observar el buzón llenándose cuando se registran muchas llegadas seguidas.

7. Almacenamiento

El almacén es una cuadrícula de 6 pasillos × 4 niveles × 5 posiciones = **120 ubicaciones** de 25 unidades cada una (capacidad total: 3000 unidades).



Tabla de ubicaciones del lote 4 (70 unidades · refrigeradoras)

Ubicación	Unidades
P1-N1-4	25
P1-N2-2	25
P1-N2-3	20

Un mismo lote ocupa ubicaciones NO contiguas
- El lote 4 (en rojo) quedó en P1-N1-4, P1-N2-2 y P1-N2-3: la primera estaba libre porque se despachó el lote anterior, y la siguiente libre estaba más adelante.
- Cualquier ubicación libre sirve: no hay fragmentación externa.
- La tabla del lote es su "tabla de páginas" (análogo a la paginación).
- Los pedidos retiran primero del lote más antiguo (FIFO) y liberan las ubicaciones que quedan vacías.
- Solo hay fragmentación interna en la última ubicación de cada lote.

Figura 4. *Un lote repartido en ubicaciones no contiguas*

- **Asignación no contigua** (análoga a la paginación): un lote ocupa las primeras ubicaciones libres que encuentre, aunque estén dispersas por pasillos y niveles. La **lista de ubicaciones de cada lote** es su "tabla de páginas".
- **Sin fragmentación externa:** cualquier ubicación libre sirve, no se exige contigüidad. Hay fragmentación interna en la última ubicación de cada lote.
- **Consumo FIFO:** los pedidos retiran primero del lote más antiguo y liberan las ubicaciones que quedan vacías (verificado por `WarehouseTest`).
- **Almacén lleno:** se guarda lo que quepa y se registra el sobrante en la bitácora; si no hay ninguna ubicación libre se lanza `WarehouseFullException`.
- Todos los métodos son `synchronized`: el almacén es compartido entre el hilo de Inventario y los trabajadores.

Existencias. Se distinguen *disponible* y *reservado*. `tryReserve` es atómico (todo o nada) bajo `stockLock`. Si un pedido se aborta, `release` devuelve la reserva; al completarse, `commit` retira físicamente las unidades.

8. Deadlock

8.1 Condiciones de Coffman

Condición	En REDXela
Exclusión mutua	Semáforos
Retención y espera	El pedido retiene los recursos obtenidos mientras pide el siguiente
Sin expropiación	Los recursos no se quitan; solo se sueltan voluntariamente
Espera circular	Dos pedidos piden los mismos recursos en orden inverso

8.2 Prevención (modo normal)

Se rompe la **espera circular**: todos los pedidos piden sus recursos en un **orden global fijo** (el orden de declaración de `ResourceType`). Si todos piden en el mismo orden, no puede formarse un ciclo. Es la técnica de menor costo: no requiere detección ni recuperación.

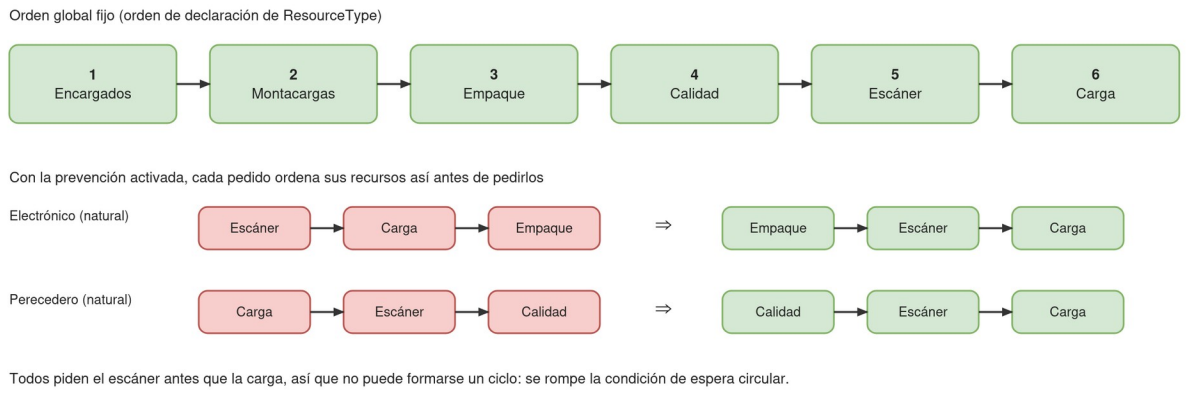


Figura 5. *Prevención: todos los pedidos piden los recursos en el mismo orden global*

Verificado en `ResourceManagerTest.sameOrderNeverDeadlocks` y en ejecución real (los mismos tres pedidos que bloquean el sistema se completan con la prevención activada).

8.3 Provocar el deadlock

Con la prevención desactivada, cada pedido pide en el orden natural de su tipo. El botón **Provocar deadlock** crea tres pedidos Exprés:

Pedido	Pide	Resultado
A (electrónico)	Escáner → Carga	Retiene el escáner, espera carga
B, C (perecederos)	Carga → Escáner	Cada uno retiene una de las 2 áreas de carga, esperan el escáner

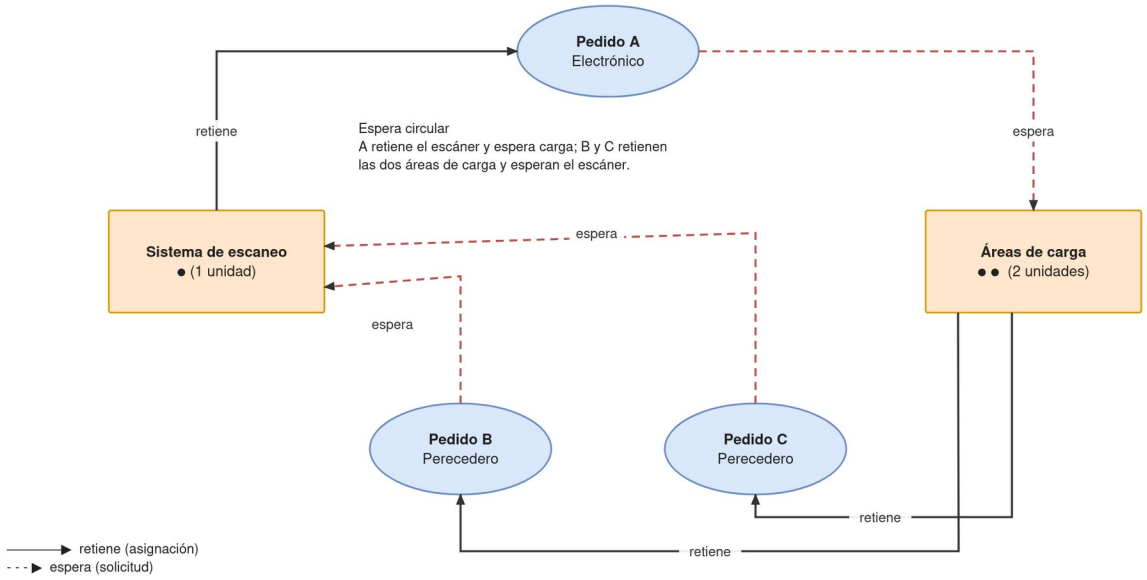


Figura 6. *Grafo de asignación y espera del escenario de deadlock*

Se necesitan **tres** pedidos porque hay 2 áreas de carga: para que A espere, ambas deben estar retenidas. Una pausa de 1.5 s entre una adquisición y la siguiente (`THINK_UNSAFE_MS`) hace el escenario reproducible. Se detiene el generador para que no interfiera. Antes de crear los pedidos se verifica que haya existencias (se elige el producto con más stock de cada tipo); si no alcanzan, se responde con un error que pide registrar una llegada, para que el escenario no falle en silencio dejando los pedidos esperando existencias.

Alcance de la prevención. Solo funciona si TODOS los pedidos en curso piden en el orden global. Un pedido que ya fue despachado con la prevención activada y otro que se despacha después con ella desactivada pueden formar un ciclo juntos (se observó en las pruebas). Por eso el botón desactiva la prevención antes de crear los pedidos del escenario.

8.4 Detección

`ResourceManager.detectDeadlock()` aplica el algoritmo de detección para recursos con varias instancias, basado en las matrices Asignación, Solicitud y Disponible:

1. `Disponible = capacidad − retenido`.
2. Una operación puede **terminar** si no espera nada o lo que espera está disponible; al terminar devuelve lo que retiene al conjunto disponible.
3. Se repite hasta que ninguna más pueda terminar. Las que retienen algo y no pudieron terminar están en deadlock.
4. Se descartan las que solo están bloqueadas **indirectamente** (retienen algo que nadie del grupo necesita): no forman parte del ciclo.

Se usa este algoritmo y no una simple búsqueda de ciclos en un grafo, porque con recursos de varias unidades un ciclo no implica deadlock: otro dueño ajeno al ciclo podría liberar una unidad. El detector se ejecuta con cada foto de estado que pide la interfaz.

La interfaz muestra los recursos involucrados, quién retiene qué, quién espera qué, quién bloquea a quién y una explicación textual de la espera circular.

8.5 Recuperación

El usuario elige la operación que continúa. Para las demás:

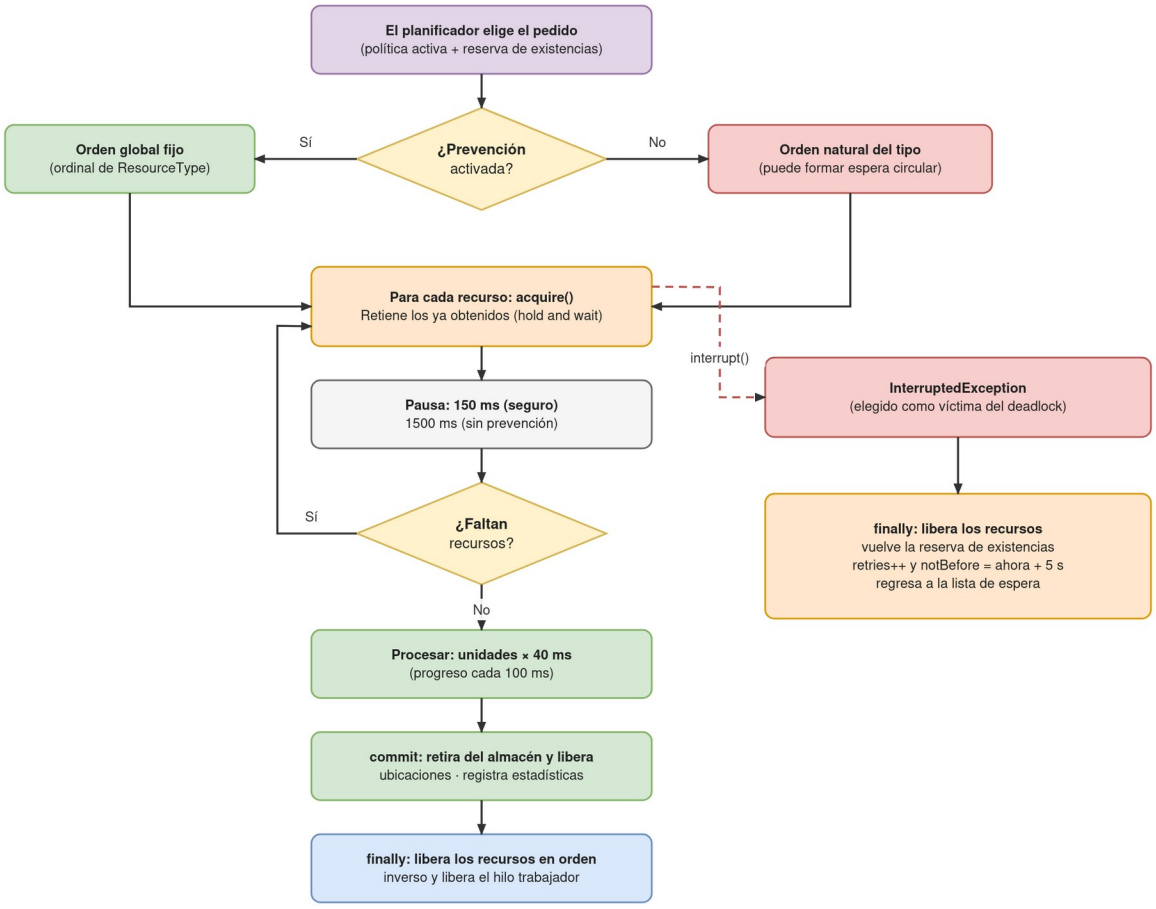


Figura 7. *Flujo de un hilo trabajador, incluida la recuperación por interrupción*

1. Se llama a `Thread.interrupt()` sobre su hilo trabajador.
2. `Semaphore.acquire()` es interrumpible y lanza `InterruptedException`.
3. El trabajador captura la excepción, **libera todos sus recursos** (`finally`) y devuelve su reserva de existencias.
4. El pedido regresa a la lista de espera con un **enfriamiento de 5 s**, para no volver a bloquearse de inmediato.
5. El ganador obtiene el recurso que esperaba y continúa.

Se interrumpe a todas las demás operaciones del grupo (no solo a una) porque con tres involucradas basta con que continúe una para romper el ciclo, y así el resultado es predecible.

9. Manejo de excepciones

Situación	Tratamiento
`InterruptedException` en un trabajador	Es la señal de aborto (víctima de deadlock): libera recursos y reencola
Error inesperado al procesar un pedido	Se registra, se liberan recursos y existencias y el pedido vuelve a la espera
Mensaje defectuoso en Inventario	Se registra y el hilo consumidor sigue vivo
Almacén lleno	`WarehouseFullException`; se guarda lo posible y se informa
Datos inválidos (cliente, producto, cantidad)	`RedxelaException` → HTTP 400 con `{"error": "..."}`
Resolver sin deadlock activo	`IllegalStateException` → HTTP 409
Tarea periódica del generador	Se captura todo: una excepción no cancela el `ScheduledExecutor`
Hilo reutilizado del pool tras una interrupción	`Thread.interrupted()` en `finally` limpia la bandera

10. Generador de pedidos

Un `ScheduledExecutorService` crea un pedido cada *N* ms (configurable) sin límite de cantidad. El nivel se elige con pesos (10/15/30/25/20 % para los niveles 1 a 5); el cliente, al azar entre los del nivel; el tipo de mercancía y 1 o 2 de sus productos, también al azar, con cantidades pequeñas más probables. Opcionalmente genera camiones con mercadería del producto que tenga menos existencias.

11. Pruebas

```
mvn test
```

Prueba	Verifica
`detectsDeadlockWithOppositeOrderAndResolvesItByInterrupt`	El detector identifica 3 operaciones en deadlock y la interrupción lo resuelve
`sameOrderNeverDeadlocks`	Con el mismo orden de recursos no hay deadlock
`lotCanBeSplitAcrossNonContiguousSlots`	Un lote se reparte en ubicaciones no contiguas
`storesWhatFitsAndReportsTheRest`	Almacén lleno: guarda lo posible, informa el sobrante y lanza excepción
`removeFreesSlots`	Retirar mercadería libera ubicaciones

Además se verificó manualmente: flujo normal con el generador, deadlock y su resolución, prevención activada con los mismos pedidos, y errores de validación.